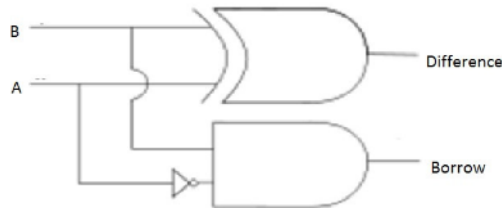


Circuit Simulation Lab: 3

The purpose of this experiment is to introduce the design of simple combinational circuits, in this case half subtractor, and full subtractor



Half Subtractor

Dataflow modeling

```
//dataflow
```

```
module half_sub (
```

```
input a , b ,
```

```
output borrow , difference );
```

```
assign difference = a ^ b ;
```

```
assign borrow = ~a & b ;
```

```
endmodule
```

Tb

```
module half_sub_tb ;
```

```
reg a , b ;
```

```
wire borrow , differnce ;
```

```
half_sub h1 (.a(a),.b(b),.borrow(borrow),.difference(difference));
```

```
initial begin
```

```
$dumpfile ("half_sub.vcd");
```

```
$dumpvars(0,half_sub_tb);
```

```
repeat (20) begin
```

```
a = $random %2 ;
```

```
b = $random %2;
```

```
#20;
```

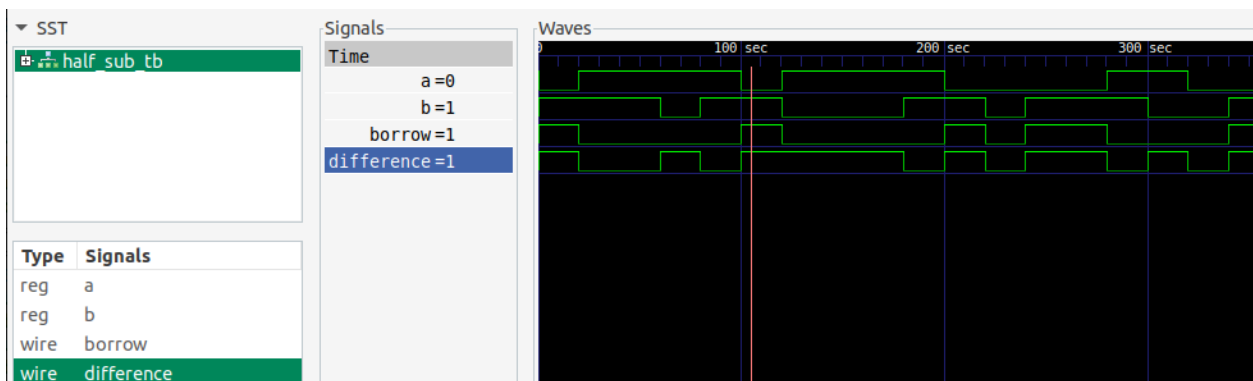
```
$display ("a = %d , b = %d , borrow = %d , difference = %d " , a,b,borrow,difference);
```

```
end
```

```
$finish;
```

```
end
```

```
endmodule
```



Behavior modeling

//behavioural modeling (BM)

```
module half_sub_BM(
input a , b ,
output reg borrow ,difference);
```

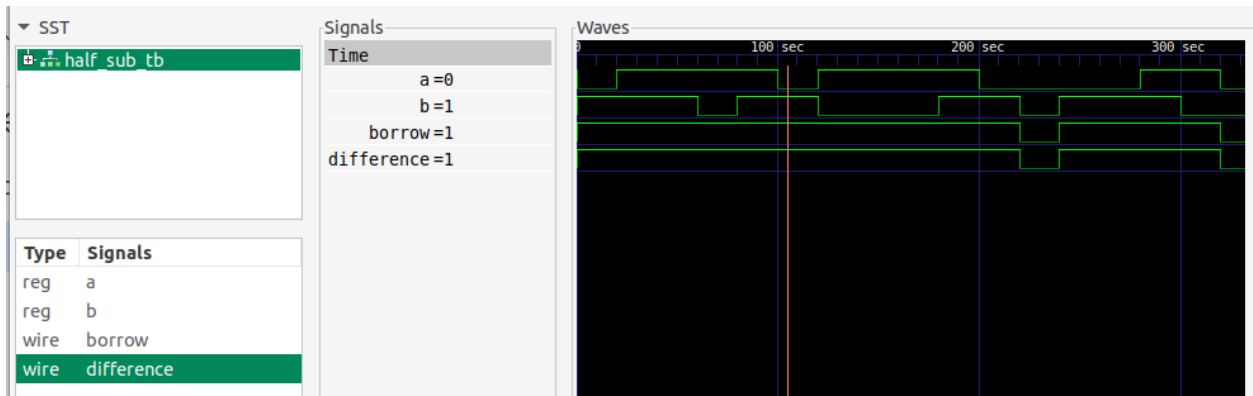
```
always @( a | b ) begin
```

```
  borrow = ~a & b;
```

```
  difference = a ^ b ;
```

```
end
```

```
endmodule
```



Structural modeling

XOR + AND +NOT = half sub

```
module xor1(
```

```
input a , b,
```

```
output c );
```

```
assign c = a ^ b ;  
endmodule
```

```
module not1 (  
input a ,  
output not_a);
```

```
assign not_a = ~ a;  
endmodule
```

```
module and1(  
input a , b ,  
output c );
```

```
assign c = a & b ;  
endmodule
```

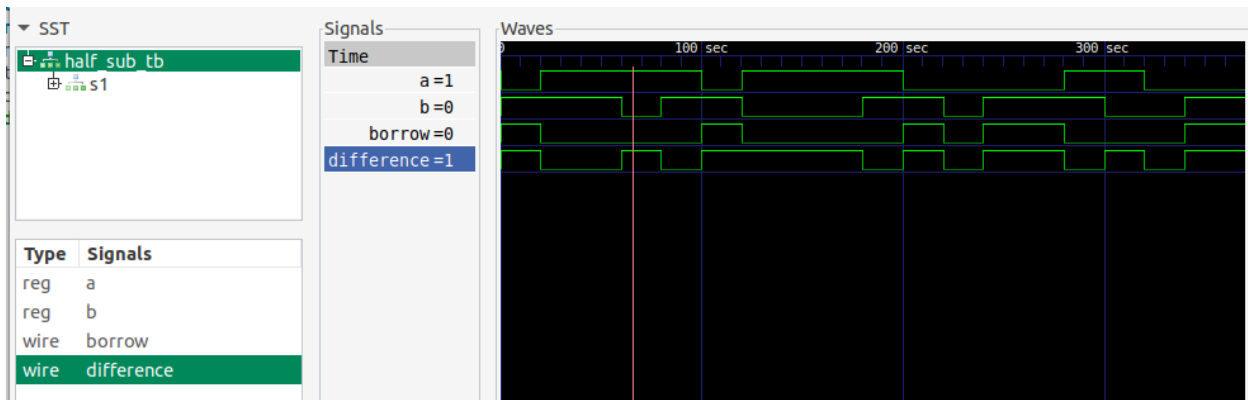
```
//structural modeling  
module half_sub_SM(  
input a , b ,  
output borrow ,difference);  
wire not_a;  
xor1 x1 (.a(a),.b(b),.c(difference));  
not1 n1 (.a(a),.not_a(not_a));  
and1 a1 (.a(not_a),.b(b),.c(borrow));  
endmodule
```

```
module half_sub_tb ;  
reg a , b ;  
wire borrow , differnce ;
```

```
half_sub_SM s1 (.a(a),.b(b),.borrow(borrow),.difference(difference));
```

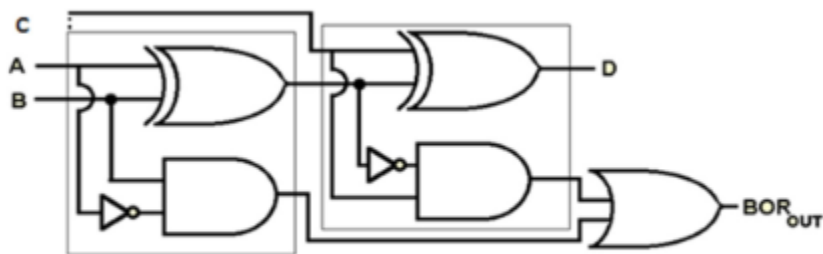
```
initial begin  
$dumpfile ("half_sub_SM.vcd");  
$dumpvars(0,half_sub_tb);  
repeat (20) begin  
a = $random %2 ;  
b = $random %2;  
#20;  
$display ("a = %d , b = %d , borrow = %d , difference = %d " , a,b,borrow,difference);  
end  
$finish;  
end
```

endmodule



```
vlsi_dev_box@vlsi-box:~/Documents/NIELIT_ASSIGNMENTS/EXP_NO_3$ iverilog -o out x
or1.v not1.v and1.v half_sub_SM.v half_sub_tb.v
vlsi_dev_box@vlsi-box:~/Documents/NIELIT_ASSIGNMENTS/EXP_NO_3$ vvp out
VCD info: dumpfile half_sub_SM.vcd opened for output.
a = 0 , b = 1 , borrow = 1 , difference = 1
a = 1 , b = 1 , borrow = 0 , difference = 0
a = 1 , b = 1 , borrow = 0 , difference = 0
a = 1 , b = 0 , borrow = 0 , difference = 1
a = 1 , b = 1 , borrow = 0 , difference = 0
a = 0 , b = 1 , borrow = 1 , difference = 1
a = 1 , b = 0 , borrow = 0 , difference = 1
a = 1 , b = 0 , borrow = 0 , difference = 1
a = 1 , b = 0 , borrow = 0 , difference = 1
a = 1 , b = 1 , borrow = 0 , difference = 0
a = 0 , b = 1 , borrow = 1 , difference = 1
a = 0 , b = 0 , borrow = 0 , difference = 0
a = 0 , b = 1 , borrow = 1 , difference = 1
```

FULL ADER



Full Subtractor

Structural modeling

```
module full_sub_ST(
input a , b , bin ,
output difference , borrow );
wire h1_diff , h1_borrow;
wire h2_borrow;
```

```

half_sub h1 (.a(a),.b(b),.difference(h1_diff),.borrow(h1_borrow));
half_sub h2 (.a(bin),.b(h1_diff),.difference(difference),.borrow(h2_borrow));
or1 o1 (.a(h1_borrow),.b(h2_borrow),.out1(borrow));
Endmodule

```

```

//dataflow
module half_sub (
input a , b ,
output borrow , difference );

```

```

assign difference = a ^ b ;
assign borrow = ~a & b ;
endmodule

```

```

module or1(
input a , b ,
output out1);
assign out1= a | b ;
Endmodule

```

```

module full_sub_tb ;
reg a , b , bin;
wire borrow , differnce ;

```

```

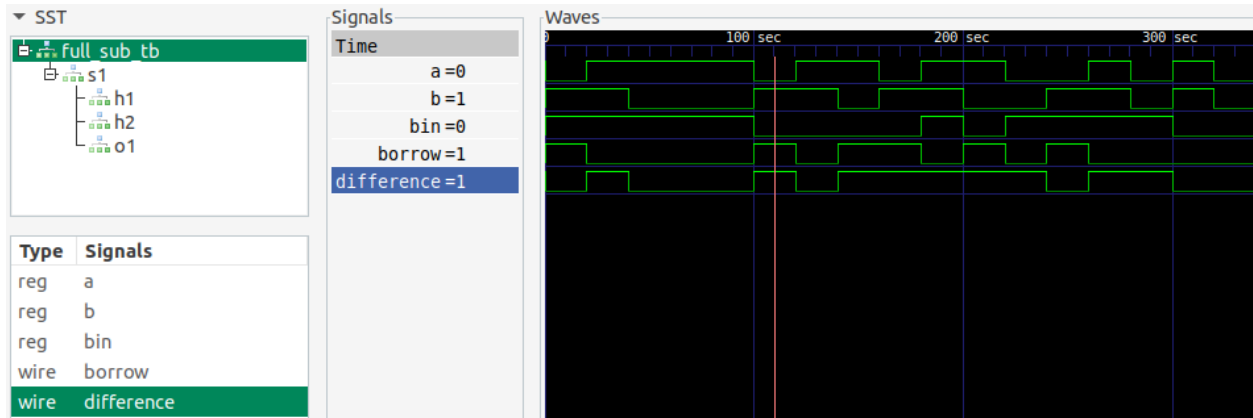
full_sub_ST s1 (.a(a),.b(b),.bin(bin),.borrow(borrow),.difference(difference));

```

```

initial begin
$dumpfile ("full_sub_ST.vcd");
$dumpvars(0,full_sub_tb);
repeat (20) begin
a = $random %2 ;
b = $random %2;
bin = $random %2;
#20;
$display ("a = %d , b = %d ,bin = %d , borrow = %d , difference = %d " ,
a,b,bin,borrow,difference);
end
$finish;
end
endmodule

```



```

vlsi_dev_box@vlsi-box:~/Documents/NIELIT_ASSIGNMENTS/EXP_NO_3/full_subs$ iverilog
g -o out or1.v full_sub_tb.v full_sub_ST.v half_sub.v
vlsi_dev_box@vlsi-box:~/Documents/NIELIT_ASSIGNMENTS/EXP_NO_3/full_subs$ vvp out
VCD info: dumpfile full_sub_ST.vcd opened for output.
a = 0 , b = 1 ,bin = 1 , borrow = 1 , difference = 0
a = 1 , b = 1 ,bin = 1 , borrow = 0 , difference = 1
a = 1 , b = 0 ,bin = 1 , borrow = 0 , difference = 0
a = 1 , b = 0 ,bin = 1 , borrow = 0 , difference = 0
a = 1 , b = 0 ,bin = 1 , borrow = 0 , difference = 0
a = 0 , b = 1 ,bin = 0 , borrow = 1 , difference = 1

```